



第5章 CPU调度

- 本章内容
 - 1. 基本概念
 - 2. 调度准则
 - 3. 调度算法
 - 4. 多处理器调度
 - 5. 案例





复习

■ 处理机的三级调度：

■ 作业调度（长程调度，高级调度）

- 从外存后备队列的作业中选择一个或多个，分配内存、IO设备等必要资源，建立进程

■ 进程调度（短程调度，低级调度）

- 从就绪队列中选取一个进程，将CPU分配给它

■ 交换调度（中程调度，中级调度）

- 将内存中暂时不用的进程移到外存，以腾出空间给其它进程使用，或将之前移出的进程从外存读入内存

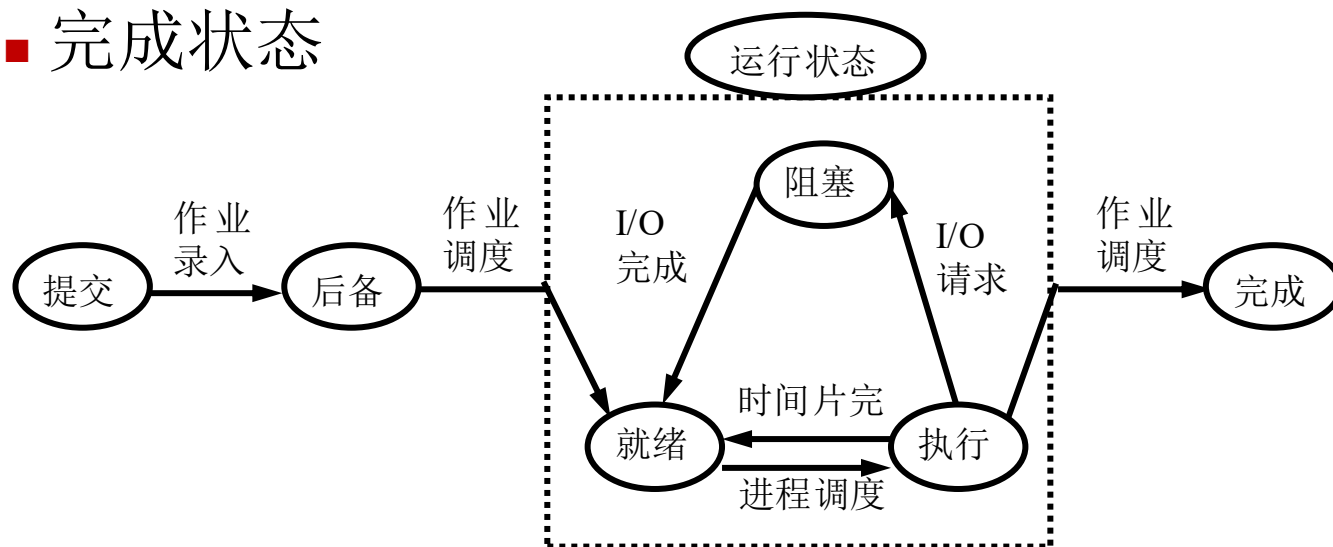




复习

■ 作业从提交到完成要经历四种状态

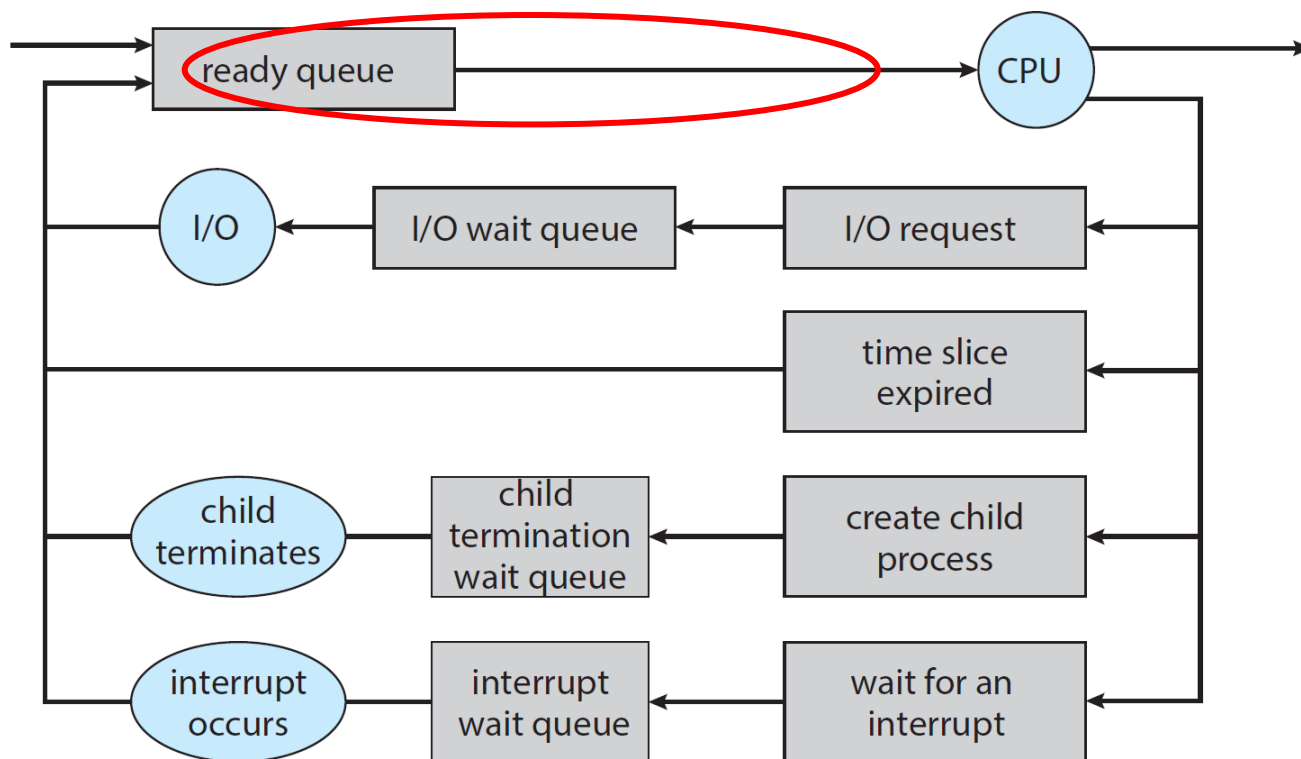
- 提交状态：作业向系统外存输入
- 后备状态：系统为作业建立作业控制块，并插入到后备作业队列中等待调度
- 运行状态：作业在内存中执行，表现为一个进程
- 完成状态





复习

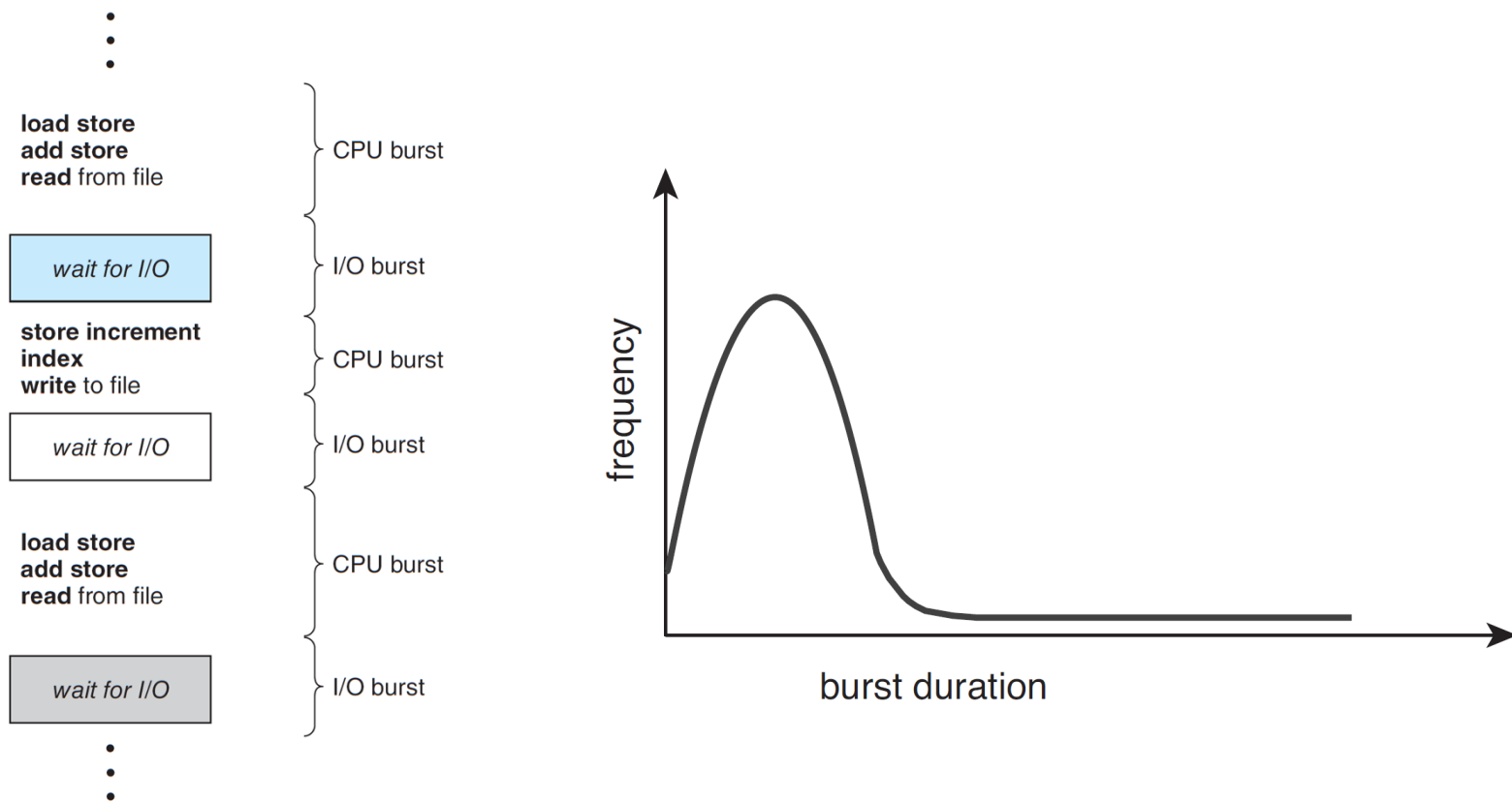
- 作业控制块与作业调度
 - 略，有兴趣者自学
- 本章重点：进程调度





进程执行的规律

- 由CPU执行和I/O等待周期组成
 - 大量测试表明CPU区间长度曲线呈现为大量短CPU区间和少量长CPU区间





CPU调度程序

- 当CPU空闲时，CPU调度程序从就绪队列中选择一个进程来执行
- CPU调度可能发生在下述四种情况下
 - 从运行状态转到等待状态
 - 从运行状态转到就绪状态
 - 从等待状态转到就绪状态
 - 进程终止





抢占及非抢占调度

- 非抢占方式：又称非剥夺方式
 - 一旦将处理机分配给某进程，便一直执行，直到其完成或阻塞，才把处理机分配给其他进程
 - 特点：简单，系统开销小，但无法处理紧急任务
- CPU调度仅发生在两种情况下
 - 从运行状态转到等待状态
 - 进程终止





抢占及非抢占调度

- 抢占方式：又称剥夺方式
 - 不等进程主动释放CPU，利用中断或系统调用的机会重新调度
 - 抢占原则有：优先权、时间片
- CPU调度发生在所有四种情况下
 - 从运行状态转到等待状态
 - 从运行状态转到就绪状态
 - 从等待状态转到就绪状态
 - 进程终止





抢占及非抢占调度

■ 进一步讨论

■ 抢占有不同力度，对于分时系统

- 只在定时器中断，时间片到期的情况下重新调度（从运行转就绪）
- 除了定时器中断，进程创建、唤醒等情况下，也借机重新调度（从等待转就绪）
- 甚至利用每次中断（或系统调用）的机会重新调度（从运行转就绪）

■ 对于批处理系统

- 进程创建、唤醒等情况下，可借机重新调度（从等待转就绪）





分派程序

- 分派程序用来将对CPU的控制交给由短程调度程序选择的进程，其功能包括
 - 切换上下文
 - 切换到用户态
 - 跳转到用户程序的适当位置并重新运行之
- 分派延迟
 - 分派程序终止一个进程的运行并启动另一个进程运行所花的时间
 - 也称为调度时间





第5章 CPU调度

- 本章内容
 - 1. 基本概念
 - 2. 调度准则
 - 3. 调度算法
 - 4. 多处理器调度
 - 5. 案例





调度准则

- 几种主要的评价准则
 - **CPU利用率**：使CPU尽可能的忙碌
 - **吞吐量**：单位时间内运行完的进程数
 - **周转时间**：进程从**提交**到运行**结束**的时间间隔
 - **等待时间**：进程在**就绪队列**中等待调度的时间**总和**
 - **响应时间**：从提交请求到系统首次产生响应所用的时间





周转时间

- 周转时间：从作业提交到完成的时间间隔

- 平均周转时间（ T_i 为作业 i 的周转时间）

$$T = (T_1 + T_2 + \dots + T_n) / n$$

- 带权周转时间：作业周转时间与实际运行时间的比

- 平均带权周转时间（ W_i 为作业 i 的带权周转时间）

$$W = (W_1 + W_2 + \dots + W_n) / n$$





周转时间及带权周转时间例

■ 各作业感受如何？

作业	提交时间	运行时间	完成时间	周转时间	带权周转时间	等待时间
A	8	2	11	3	1.5	1
B	8.2	0.1	11.2	3	30	2.9
C	9	0.2	10.2	1.2	6	1





第5章 CPU调度

- 本章内容
 - 1. 基本概念
 - 2. 调度准则
 - 3. 调度算法
 - 4. 多处理器调度
 - 5. 案例





先来先服务

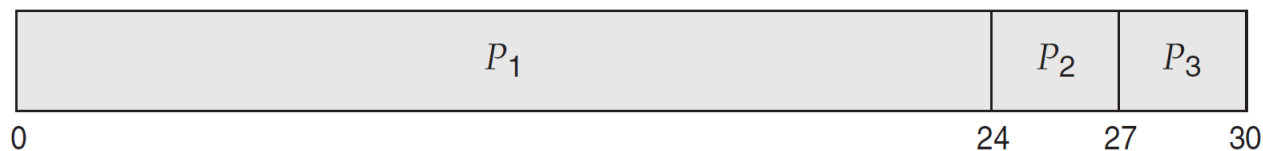
■ FCFS: 先请求CPU的进程先分配到CPU

■ 例1

- 有3个进程

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- 假设进程的到达顺序为: P_1, P_2, P_3 , 甘特图如下:



- 进程等待时间: $P_1 = 0; P_2 = 24; P_3 = 27$
- 平均等待时间: $(0 + 24 + 27) / 3 = 17$



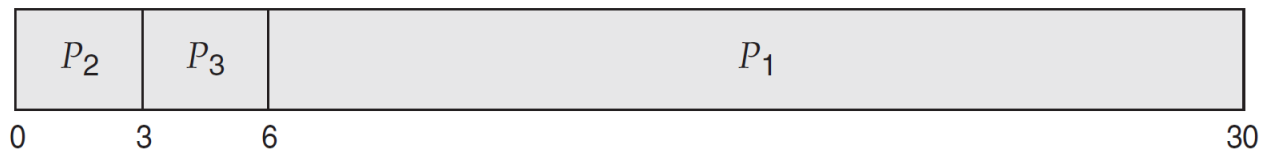


先来先服务

■ FCFS: 先请求CPU的进程先分配到CPU

■ 例2

- 假设进程到达的顺序为: P2, P3, P1, 甘特图如下:



- 进程等待时间: $P1 = 6$; $P2 = 0$; $P3 = 3$
- 平均等待时间: $(6 + 0 + 3) / 3 = 3$
- 性能优于前一种情况
- 车队效应 (护航效应): 短进程排在长进程后面





先来先服务

■ 习题

- 设有4道作业，它们的提交时间及执行时间如下表，按先来先服务算法进行调度，试计算4个作业的平均周转时间和平均带权周转时间。（时间单位：小时）

作业	提交时间	估计运行时间
1	10	2
2	10.2	1
3	10.4	0.5
4	10.5	0.3





先来先服务

■ 习题

作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
1	10	2	10	12	2	1
2	10.2	1	12	13	2.8	2.8
3	10.4	0.5	13	13.5	3.1	6.2
4	10.5	0.3	13.5	13.8	3.3	11

平均周转时间: $T=(2.0+2.8+3.1+3.3)/4=2.8$

平均带权周转时间: $W=(1+2.8+6.2+11)/4=5.25$





先来先服务

■ 特点

- 算法简单，易于实现
- 但不利于短作业及I/O繁忙型作业
- 假定不抢占，仅适用于批处理系统，仅适用于计算任务





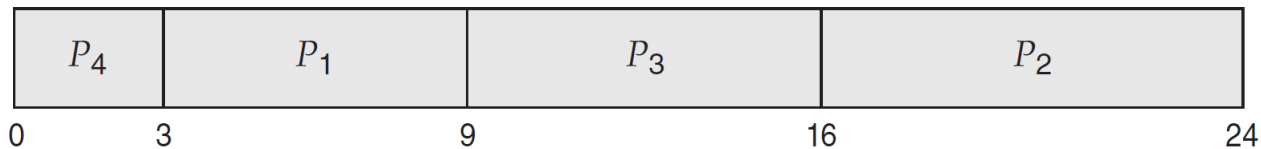
最短作业优先

■ SJF: 将CPU赋予估计运行时间最短的进程

■ 例1: 非抢占

■ 有4个进程

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3



■ 进程等待时间: $P_1 = 3$; $P_2 = 16$; $P_3 = 9$; $P_4 = 0$

■ 平均等待时间: $(3 + 16 + 9 + 0) / 4 = 7$





最短作业优先

■ 习题：非抢占

作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
1	10	2	10	12	2	1
2	10.2	1	12.8	13.8	3.6	3.6
3	10.4	0.5	12.3	12.8	2.4	4.8
4	10.5	0.3	12	12.3	1.8	6

平均周转时间： $T = (2.0 + 1.8 + 2.4 + 3.6) / 4 = 2.45$

平均带权周转时间： $W = (1 + 6 + 4.8 + 3.6) / 4 = 3.85$





最短作业优先

■ 特点

- 性能较好，可证明该算法平均等待时间最短
- 但对长作业不利，未考虑作业的紧迫程度
- 运行时间靠估计：程序员自行预估，提交时管理员审核录入，或根据之前运行情况估算
- 可增加抢占机制：最短剩余时间优先，SRT
 - 基于时间片的抢占无意义，得到CPU的进程越运行优势越明显
 - 仍然只适用于批处理系统，主要是在新的进程加入竞争的情况下，如进程创建、唤醒环节





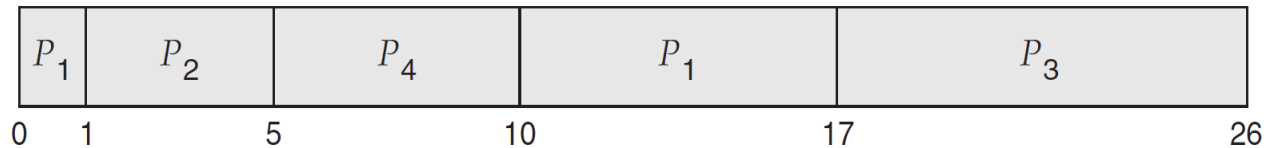
最短剩余时间优先

■ SRT: 抢占的最短作业优先

■ 例2: 抢占

- 有4个进程

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



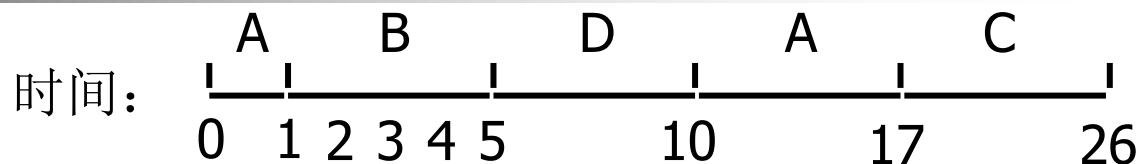
- 平均等待时间: $[(10-1)+(1-1)+(17-2)+(5-3)] / 4 = 6.5$
- 若采用非抢占: 7.75





最短剩余时间优先

■ 习题：抢占



作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
A	0	8	0	17	17	2.125
B	1	4	1	5	4	1
C	2	9	17	26	24	2.67
D	3	5	5	10	7	1.4

平均周转时间： $T = (17 + 4 + 24 + 7) / 4 = 13$

平均带权周转时间： $W = (2.125 + 1 + 2.67 + 1.4) / 4 = 1.8$





时间片轮转调度

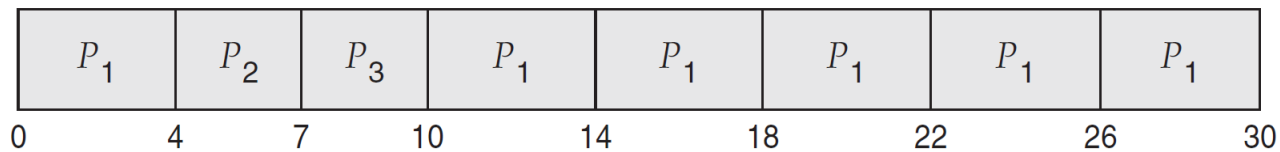
■ RR: 时间片用完后插入就绪队列末尾

■ 例

- 有3个进程

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- 假定时间片4ms



- 进程等待时间: $P_1 = 6$; $P_2 = 4$; $P_3 = 7$
- 平均等待时间: $(6 + 4 + 7) / 3 = 5.66$





时间片轮转调度

■ 习题1：时间片为1

- A、B、C、D、E要求运行时间依次为3、6、4、5、2，到达时间依次为0、1、2、3、4

0: A运行;	10: D运行, ECB等待;
1: B运行, A等待;	11: E运行, CBD等待;
2: A运行, CB等待;	12: C运行, BD等待;
3: C运行, BDA等待;	13: B运行, DC等待;
4: B运行, DAEC等待;	14: D运行, CB等待;
5: D运行, AECB等待;	15: C运行, BD等待;
6: A运行, ECBBD等待;	16: B运行, D等待;
7: E运行, CBD等待;	17: D运行, B等待;
8: C运行, BDE等待;	18: B运行, D等待;
9: B运行, DEC等待;	19: D运行,





时间片轮转调度

■ 习题1：时间片为1

作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
A	0	3	0	7	7	2.33
B	1	6	1	19	18	3
C	2	4	3	16	14	3.5
D	3	5	5	20	17	3.4
E	4	2	7	12	8	4

平均周转时间： $T = (7 + 18 + 14 + 17 + 8) / 5 = 12.8$

平均带权周转时间： $W = (2.33 + 3 + 3.5 + 3.4 + 4) / 5 = 3.246$





时间片轮转调度

■ 习题2：时间片为4

- A、B、C、D、E要求运行时间依次为3、6、4、5、2，到达时间依次为0、1、2、3、4

0: A运行，BCD依次到达；

3: B运行，CD等待，后E到达；

7: C运行，DEB等待；

11: D运行，EB等待；

15: E运行，BD等待；

17: B运行，D等待；

19: D运行

A、B、C、D、E开始时间依次为0、3、7、11、15；
结束时间依次为3、19、11、20、17





时间片轮转调度

■ 习题2：时间片为4

作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
A	0	3	0	3	3	1
B	1	6	3	19	18	3
C	2	4	7	11	9	2.25
D	3	5	11	20	17	3.4
E	4	2	15	17	13	6.5

平均周转时间： $T = (3 + 18 + 9 + 17 + 13) / 5 = 12$

平均带权周转时间： $W = (1 + 3 + 2.25 + 3.4 + 6.5) / 5 = 3.23$





时间片轮转调度

■ 时间片大小的影响

- 太大则无法保证响应速度

- 极端情况：时间片极长，则退化为FCFS

- 太小则进程调度频繁，系统开销增加

- 一般为10-100ms，上下文切换一般少于10us

■ 要考虑的因素

- 系统对响应时间的要求

- 就绪队列中的进程数目

- 系统处理能力

注意：一个系统中，时间片并不总是固定长度，如何根据进程情况调整时间片长度，是现代调度算法的核心





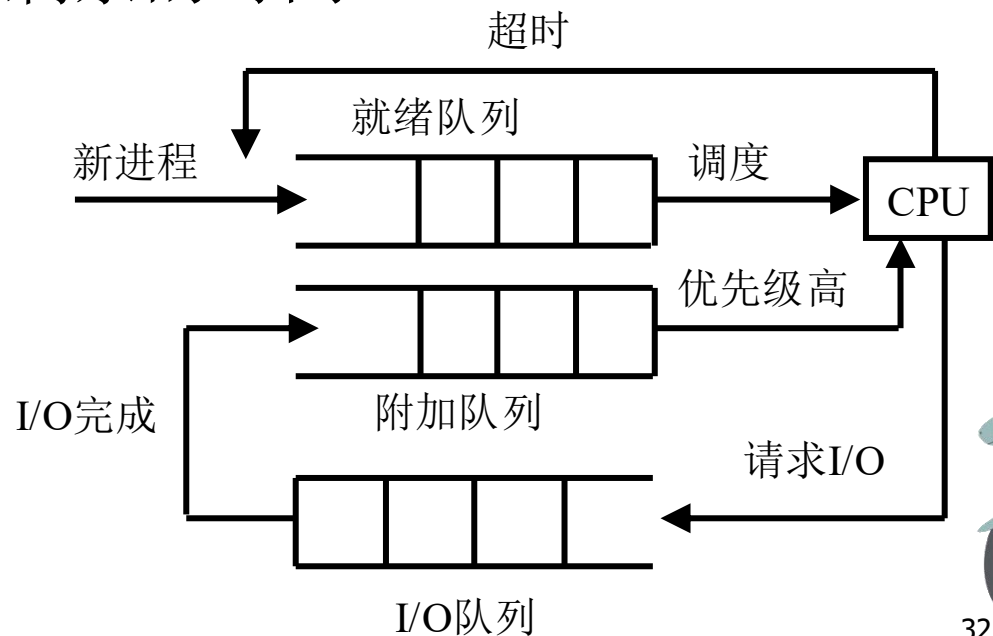
时间片轮转调度

■ 对I/O型进程的奖励（常见做法）

- 进程因I/O阻塞，唤醒后进入附加队列，附加队列的优先级高于就绪队列
- 当进程从附加队列被调度时，运行时间不超过上次阻塞前时间片剩余的时间

原因：在就绪队列里循环多轮的，都是计算任务，老油条，不在乎多等一会

刚唤醒的往往是因为用户操作或网络传输，对响应速度更敏感





优先级调度

- 优先级调度：每个进程关联一个优先级数字
 - CPU分配给最高优先级进程，相同则按FCFS
 - 某些系统中数字越大优先级越高，另一些相反

- 例1

- 有5个进程
- 数小优先级高

例 1

■ 有5个进程

■ 数小优先级高

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

P_2	P_5	P_1	P_3	P_4	
0	1	6	16	18	19

- 平均等待时间： $(6 + 0 + 16 + 18 + 1) / 5 = 8.2$





优先级调度

- SJF可被看做优先级调度，其优先级是预测的下一个CPU执行时间的倒数
- 两种方案
 - 可抢占式
 - 不可抢占式
- 需警惕：饥饿，低优先级进程永远无法执行
 - 解决方案：老化，随着时间推移，增加等待进程的优先级



优先级调度

■ 优先级分为两种

- 静态：创建进程时确定，运行期间不改变
 - 确定依据：进程类型、对资源的需求、紧迫程度
- 动态：创建进程时确定一个优先级，运行过程中再根据情况的变化调整

- 确定原则有：占用CPU时间，等待时间

- 例：
$$\text{优先数} = \frac{\text{CPU使用时间}}{2} + \text{基本优先数}$$

运行越久，优先数越大，优先级越低

- CPU使用时间衰减函数：

$$\text{Decay}(\text{CPU使用时间}) = \frac{\text{CPU使用时间}}{2}$$





优先级调度

■ 优先级调度结合时间片轮转

- 调度最高优先级的进程
- 若优先级相同，则按RR方式

■ 例2

- 有5个进程
- 数小优先级高
- 时间片为2

	<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
	P_1	4	3
	P_2	5	2
	P_3	8	2
	P_4	7	1
	P_5	3	3

0	7	9	11	13	15	16	20	22	24	26	27
P ₄		P ₂	P ₃	P ₂	P ₃	P ₂	P ₃	P ₁	P ₅	P ₁	P ₅

- 平均等待时间: $(22 + 11 + 12 + 0 + 24) / 5 = 13.8$





高响应比优先调度

- 动态优先级的一种，对SJF和FCFS的综合

- 响应比定义

$$\begin{aligned}\text{响应比} &= \frac{\text{响应时间}}{\text{估计运行时间}} = \frac{\text{等待时间} + \text{估计运行时间}}{\text{估计运行时间}} \\ &= 1 + \frac{\text{作业等待时间}}{\text{估计运行时间}}\end{aligned}$$

- 特点

- 有利于短作业：等待时间相同，短作业优先
- 考虑等待时间：运行时间相同，等待久的优先





高响应比优先调度

■ 习题

- A、B、C、D、E五个进程，到达时间为0、1、2、3、4，要求运行时间为3、6、4、5、2，计算平均周转时间和平均带权周转时间，不抢占

0: A运行，BCD依次到达

3: $r_B = 1 + 2/6$, $r_C = 1 + 1/4$, $r_D = 1$; B先运行

9: $r_C = 1 + 7/4$, $r_D = 1 + 6/5$, $r_E = 1 + 5/2$; E先运行

11: $r_C = 1 + 9/4$, $r_D = 1 + 8/5$; C先运行

由此可知作业的运行顺序为A、B、E、C、D





高响应比优先调度

■ 习题

顺序A、B、E、C、D

作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
A	0	3	0	3	3	1
B	1	6	3	9	8	1.33
C	2	4	11	15	13	3.25
D	3	5	15	20	17	3.4
E	4	2	9	11	7	3.5

平均周转时间: $T = (3 + 8 + 13 + 17 + 7) / 5 = 9.6$

平均带权周转时间: $W = (1 + 1.33 + 3.25 + 3.4 + 3.5) / 5 = 2.496$





多级队列调度

- **MQ**: 将就绪队列分为多个独立队列
 - 每个队列有自己的调度算法
 - 各队列优先级不同
 - 可以绝对遵守优先级，高的必须清空，低的才有机会
 - 也可以按优先级分配各队列的时间占比
 - 根据进程属性，一个进程被永久分配到一个队列
 - 例，可以将就绪进程队列分为
 - 前台（交互式），RR，优先级高
 - 后台（批处理），FCFS，优先级低

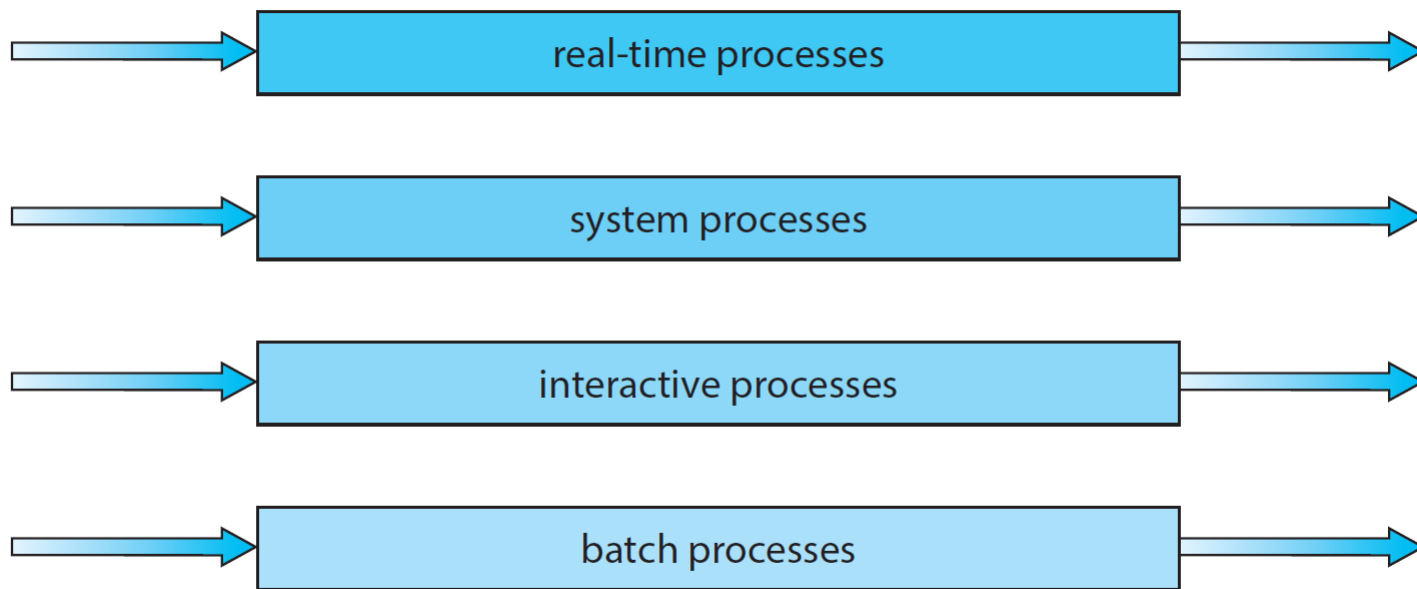




多级队列调度

■ 例

highest priority



lowest priority





多级反馈队列调度

- MFQ: 进程能在不同的队列间移动
 - 使用CPU时间过多, 会被移到更低级队列
 - 在低级队列中等待过久, 会被移到更高级队列
 - 调度程序由以下参数定义
 - 队列数
 - 每一队列的调度算法
 - 进程应进入哪个队列
 - 什么情况下升级
 - 什么情况下降级





多级反馈队列调度

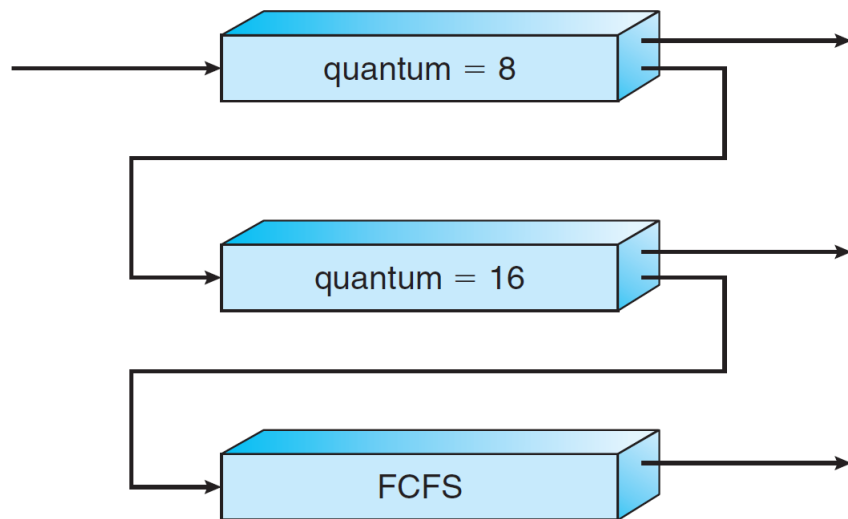
■ MFQ: 进程能在不同的队列间移动

■ 例

- 新进程放入第1队列末尾，排队调度
- 如未能在时间片内完成，转入第2队列末尾，以此类推
- 仅当前 $i-1$ 个队列均为空，才会调度第 i 队列
- 第 i 队列中的进程若执行时被抢占，被放回第 i 队列末尾

注意：一个进程不是一辈子只在这里走一趟，高开低走，悲惨落幕

前面说过，进程的执行是CPU burst和IO burst交替，此处仅涉及进程的一次CPU burst，其它调度算法也如此





多级反馈队列调度

■ 习题

- 有3个队列，时间片为1、2和4
- A、B、C、D、E五个进程，到达时间为0、1、3、4、5，要求运行时间为3、8、4、5、7

0: A运行;

1: B运行, A等待;

2: A运行, B等待;

3: C运行, BA等待;

4: D运行, BAC等待;

5: E运行, BACD等待;

6: BB运行, ACDE等待;

8: A运行, CDE等待; B等待

9: CC运行, DE等待; B等待

11: DD运行, E等待; BC等待

13: EE运行, BCD等待;

15: BBBB运行, CDE等待;

19: C运行, DEB等待;

20: DD运行, EB等待;

22: EEEE运行, B等待;

26: B运行。





多级反馈队列调度

■ 习题

作业	提交时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
A	0	3	0	9	9	3
B	1	8	1	27	26	3.25
C	3	4	3	20	17	4.25
D	4	5	4	22	18	3.6
E	5	7	5	26	21	3

平均周转时间: $T = (9 + 26 + 17 + 18 + 21) / 5 = 18.25$

平均带权周转时间: $W = (3 + 3.25 + 4.25 + 3.6 + 3) / 5 = 3.42$





多级反馈队列调度

■ 特点

- 终端型用户：大多能在一个时间片内完成，响应时间较短
- 短批处理作业用户：能在前几个队列完成，周转时间较短
- 长批处理作业用户：依次在 n 个队列中运行，不会长时间得不到处理





第5章 CPU调度

- 本章内容
 - 1. 基本概念
 - 2. 调度准则
 - 3. 调度算法
 - 4. 多处理器调度
 - 5. 案例





基本策略

- 根据多处理器的对称性
 - 非对称多处理器：不同CPU有不同的任务
 - 一些专用于内核态，一些专用于用户态
 - 使用场景多变，易出现瓶颈
 - 对称多处理器（SMP）：CPU都是对等的
 - 都可执行任何一个任务，所有CPU共享内存和I/O设备
 - 所有主流OS的选择





基本策略

- 就绪队列的共享/专属
 - 所有CPU共用一个就绪队列
 - 共享访问易冲突，须用原语等手段轮流访问，效率低下
 - 每个CPU拥有专属就绪队列
 - 主流OS的选择
 - 带来负载平衡问题
 - 还有其它优势，稍后介绍





负载均衡

- 仅限各CPU有专属就绪队列的情况
- 两种方法（通常同时采用）
 - 推迁移
 - 系统周期性检查各CPU负载，发现不平衡则迁移
 - 拉迁移
 - CPU空闲则去忙碌CPU队列中拉取
- 平衡的判别标准各不相同
 - 队列中进程数量的平衡
 - 队列中进程优先级的平衡
 - 其它





处理器亲和性

■ 核心问题

- CPU有缓存，随着程序运行被填充，达到稳定的功效（warm cache）
- 一个进程换到另一个CPU，需要重建缓存

■ 调度原则

- 同一进程优先安排在同一个CPU上
- 两种形式
 - 软亲和性：尽量，但不保证，有空闲的CPU也不放过
 - 硬亲和性：限死，不顾负载平衡
 - Linux总体遵循软亲和性，也提供硬亲和性系统调用





处理器亲和性

- 关于线程的进一步讨论
 - 同一进程的多线程共享内存，在多个CPU上同时运行，还会有公共变量的缓存同步问题，效率有折损，单个CPU上交替运行可避免折损
 - 例：4个进程，各自4个线程，4核CPU
 - 方案1：1个进程同时占据4个CPU，然后4个进程交替运行
 - 方案2：每个进程固定在一个CPU上





其它

- 有兴趣者自学
 - 超线程及其给OS的调度带来的影响
 - NUMA技术及其给OS的调度带来的影响
 - 异构多处理器（大小核）及其给OS的调度带来的影响





第5章 CPU调度

- 本章内容
 - 1. 基本概念
 - 2. 调度准则
 - 3. 调度算法
 - 4. 多处理器调度
 - 5. 案例



■ 多级反馈队列调度

- 根据进程行为和特征动态调整优先级和时间片
- 多个就绪队列，不同的优先级和时间片长度，优先级越高，时间片越短，队列内RR
- 新进程被放入最高队列中
- 如一个进程在时间片内完成或阻塞，则下次就绪时优先级不变，放入原队列
- 如一个进程在时间片内未完成或被抢占，则降低优先级，放入下一级队列





■ 调度效果

- 对于短作业或交互式作业，给予较高的优先级和较快的响应时间，提高用户体验和系统吞吐量
- 对于长作业或计算密集型作业，给予较低的优先级和较长的时间片，避免过多的上下文切换和饥饿现象
- 对于不同类型或特征的作业，根据其历史行为进行动态调整，实现自适应和公平的调度





- 基于优先级的抢占式多任务调度
 - 每个线程分配初始优先级，运行时动态调整形成当前优先级（0-31），数字越大优先级越高
 - 维护一个线程就绪表，记录哪些优先级下有就绪线程
 - 每次调度时，查找有就绪线程的最高优先级，然后在其线程链表中选择一个执行，链表中可采用轮转法或者其他方法
 - 创建、删除、阻塞、唤醒或者改变线程优先级时，会更新就绪表和线程链表





- 多种调度算法相结合
 - 优先权调度、时间片调度、交互式进程优先、多级反馈队列、抢先式调度
- 所有进程被分配到不同的调度级别，每个级别有自己的调度策略和优先级范围
 - 实时进程：最高级别，有严格时间限制的任务
 - 系统进程：如内核线程和中断处理程序
 - 交互式用户进程：如文本编辑器和浏览器
 - 批处理进程：最低级别，通常在空闲时运行





Solaris

- 优先级根据多种因素动态计算
 - 考虑进程类型、进程老化情况（即等待时间）、进程最后一次执行后的CPU使用情况等
- 任何时候只要有一个更高优先级的进程变为就绪状态，都会立即抢占当前进程
- 调度效果
 - 有效地平衡实时性、交互性和吞吐量等，能够在不同的工作负载下保持良好性能





■ 公共框架

- 每个CPU维护一个调度模块，称为runqueue
- 调度模块分为两部分
 - 一部分是公共框架，包含schedule()、load_balance()、scheduler_tick()等
 - 另一部分是封装不同策略的调度器，如针对实时进程的rt、dl调度器，针对非实时进程的cfs调度器
 - 每个调度器根据其策略实现框架要求的回调函数，如task_tick、enqueue_task、dequeue_task、pick_next_task等
 - 还要实现一些其它设置，如指定新建进程对应cfs调度器
 - 每种调度器将自己注册到公共框架中



■ 调度过程

- 当一个进程被设置`need_resched`，需要选择下一个运行进程
- 公共框架部分按照优先级，逐个调用各调度器的`pick_next_task()`，直到获取返回的进程
 - 调度器按照优先级排列依次为`stop_sched_class` > `dl_sched_class` > `rt_sched_class` > `fair_sched_class` > `idle_sched_class`
 - 当高优先级调度器中存在就绪任务时，就不会轮到低优先级调度器



- **fair_sched_class调度器（cfs调度器）**
 - 基本思想：取消预设的时间片，边运行边评估，主要评估累积运行时间
 - 基本规则
 - 引入虚拟时间，随着运行而累积，递增速度与进程优先级相关，优先级越高递增越慢
 - 本质即加权的运行时间
 - 发生进程切换的原因有两种
 - 进程主动放弃CPU，直接调用`schedule()`切换到其他进程
 - 进程被抢占





- fair_sched_class调度器（cfs调度器）
 - 在几种情况下设置抢占检查点
 - 评估各进程的虚拟时间或优先级，做出调度决策
 - 决策结论：设置need_resched，注意只决策，不执行切换
 - 几种情况：时钟中断（通常4ms或10ms周期性产生）、唤醒进程、创建进程、进程迁移
 - 后几种情况下评估优先级而非虚拟时间
 - 最短时间必须保证，超过了就要接受检查点评估
 - 根据调度延迟（一个预设的时间）计算进程的最短运行时间
 - 中断返回或系统调用返回时，检查need_resched，执行调度决策



■ fair_sched_class调度器（cfs调度器）

■ 其它

- 就绪队列使用红黑树，根据虚拟时间排序
 - 时间复杂度为 $O(\log n)$ ，比 $O(1)$ 调度器稍慢一点，但影响不大
- 弱化定时器的作用，更新进程时间的代码在多处调用，不再以定时器时长作为时间参考，时间控制更加精细
 - 还能区别统计中断以及中断下半部与进程所占用的时间
- 对进程的调度几乎只依赖于进程的虚拟时间这一属性
 - 不再需要预判进程类型而对时间片长度做复杂的计算
 - 对于某些特殊情况比如睡眠进程的奖励依旧存在，但是其扩展实现要比之前的调度器方便很多





练习

- 5.2 Explain the difference between preemptive and nonpreemptive scheduling.





练习-2

- 5.4 Consider the following set of processes, with the length of the CPU burst time given in milliseconds:

Process	Burst Time	Priority
P1	2	2
P2	1	1
P3	8	4
P4	4	2
P5	5	3

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, non-preemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of these scheduling algorithms?
- Which of the algorithms results in the minimum average waiting





选择题1

- 在分时操作系统中，进程调度经常采用 _____ 算法。
 - A. 随机
 - B. 最高优先权
 - C. 时间片轮转
 - D. 先来先服务
- _____ 优先权是在创建进程时确定的，确定之后在整个进程运行期间不再改变。
 - A. 静态
 - B. 作业
 - C. 资源
 - D. 动态





选择题2

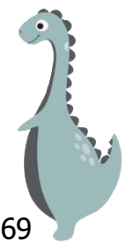
- 下述作业调度算法中，_____调度算法与作业的估计运行时间有关。
 - A. 先来先服务
 - B. 短作业优先
 - C. 时间片轮转
 - D. 多级队列





选择题3

- 现有3个同时到达的作业J1、J2和J3，它们的执行时间分别是 T_1 、 T_2 和 T_3 ，且 $T_1 < T_2 < T_3$ 。系统按单道方式运行且采用短作业优先算法，则平均周转时间是 _____。
 - A. $(3T_1 + 2T_2 + T_3)/3$
 - B. $(T_1 + T_2 + T_3)/3$
 - C. $(T_1 + 2T_2 + 3T_3)/3$
 - D. $T_1 + T_2 + T_3$
- 设有四个作业同时到达，每个作业的执行时间均为2小时，它们在一台处理器上按单道方式运行，则平均周转时间为 _____。
 - A. 8小时
 - B. 2.5小时
 - C. 5小时
 - D. 1小时





选择题4

- _____ 是指从作业提交给系统到作业完成的时间间隔。
 - A. 运行时间
 - B. 等待时间
 - C. 周转时间
 - D. 响应时间
- 在下列调度层次中，所有操作系统中都必须配置的调度层次是_____。
 - A. 作业调度
 - B. 进程调度
 - C. 交换调度
 - D. 中级调度





选择题5

- 采用时间片轮转法进行进程调度是为了_____。
 - A. 多个终端都能得到系统的及时响应
 - B. 先来先服务
 - C. 优先级较高的进程得到及时响应
 - D. 需要CPU最短的进程先做
- 假设就绪队列中有10个进程，系统将时间片设为200ms，CPU进行进程切换要花费10ms。则系统开销所占的比率约为_____。
 - A. 1% B. 5% C. 10% D. 20%





填空题1

- 在 _____ 调度算法中，按照进程进入就绪队列的先后次序来分配处理机。
- ① _____ 调度是处理机的高级调度，② _____ 调度是处理机的低级调度。





填空题2

- 进程调度算法采用时间片轮转法时，时间片过大，就会使轮转法转化为_____ 调度算法。
- 既考虑作业等待时间，又考虑作业执行时间的调度算法是_____。





考研题

■ 下列进程调度中，综合考虑进程等待时间和执行时间的是（ ） 09

A、时间片轮转调度算法 B、短进程优先调度算法
C、先来先服务调度算法 D、高响应比优先调度算法

■ 下列选项中，降低进程优先级的合理时机是____ 10

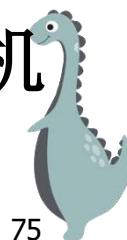
A.进程的时间片用完
B.进程刚完成I/O，进入就绪队列
C.进程长期处于就绪队列中
D.进程从就绪状态转为运行态





考研题2

- 下列选项中，满足短任务优先且不会发生饥饿现象的调度算法是____11
 - A、先来先服务 B、高响应比优先
 - C、时间片轮转 D、非抢占式短任务优先
- 若某单处理器多进程系统中有多多个就绪进程，则下列关于处理机调度的叙述中，错误的是（）12
 - A. 在进程结束时能进行处理机调度
 - B. 创建进程后能进行处理机调度
 - C. 在进程处于临界区时不能进行处理机调度
 - D. 在系统调用完成并返回用户态时能进行处理机调度





考研题3

- 一个多道批处理系统中仅有P1和P2两个作业，P2比P1晚5ms到达。它们的计算和I/O操作顺序如下：

P1: 计算60ms, I/O 80ms, 计算20ms,

P2: 计算120ms, I/O 40ms, 计算40ms。

若不考虑调度和切换时间，则完成两个作业需要的时间最少是 () 12

A. 240ms B.260ms C.340ms D.
360ms

